

Testing in **PRODUCTION** (responsibly)

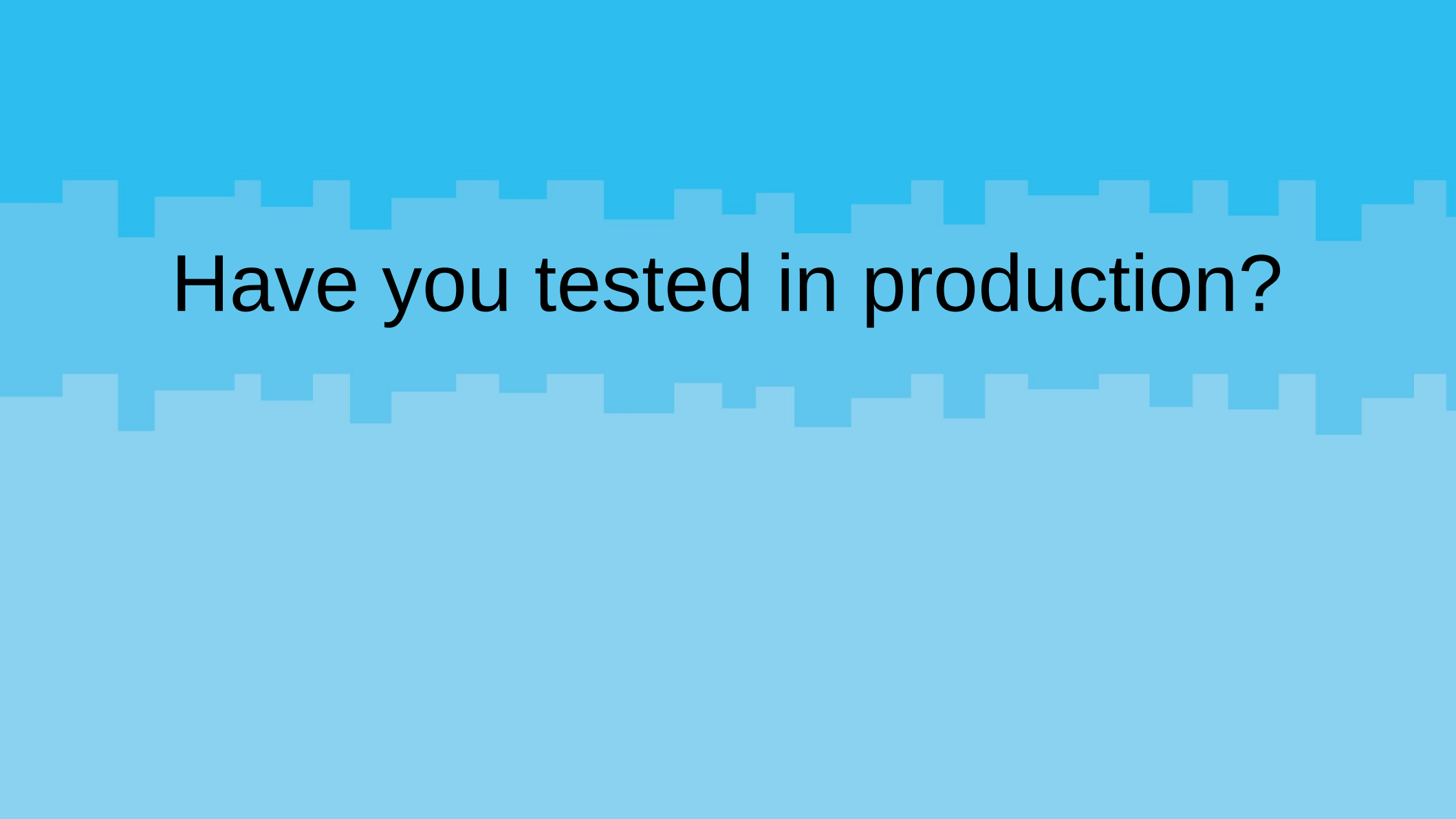
By Alex Bissessur

alex.yaml

```
---
apiVersion: v1
kind: Person
metadata:
  name: Alex Bissessur
spec:
  work:
    company: La Sentinelle
    role: Kubernetes Person
    location: Mauritius
  contact:
    website: alexbissessur.dev
    mastodon: moris.social/@AlexB
    github: github.com/xelab04
  interests:
    - Kubernetes
    - Linux
    - Free & Open Source Software
  hobbies:
    - Playing kubectl with Homelab
```

“I do fun things with
Kubernetes.”





Have you tested in production?

Everybody has a testing environment. Some people are lucky enough enough to have a totally separate environment to run production in.

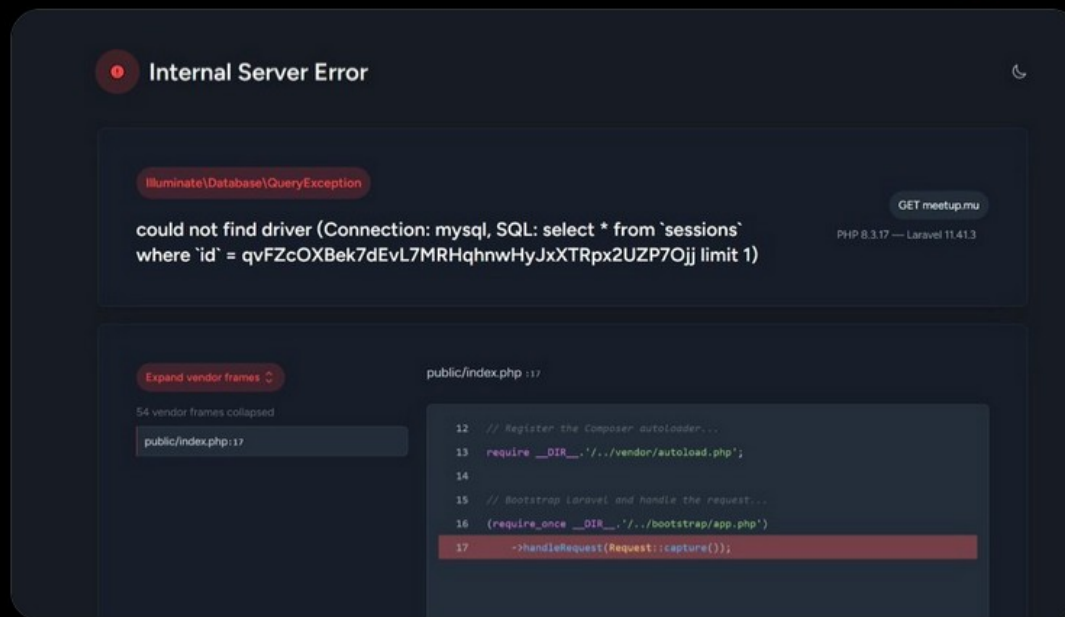


Alexandre Bissessur

@Alex_with_a_B



Technically speaking, this is a "minor inconvenience due to some faulty configuration which caused breakdown of the Laravel/MySQL interface"
Practically speaking, I believe we call this "testing in production" 🤔



10:09 PM · Mar 2, 2025 · 56 Views

View post engagements



The Olde Way

- Monolith application, annoying to deploy/update
- Single server so testing is discouraged
 - bugs **will** cause downtime and you **will** lose your job
- Rolling back issues requires manual work
- In times of crisis, priority is “fix now, investigate later”

New Infrastructure: Containerisation

- Server application is just a container
- Containers are treated as cattle, not pets
 - Can (meant to) be brought up
 - Can (meant to) be destroyed
- Containers are fluid

Traditionally:



Now:

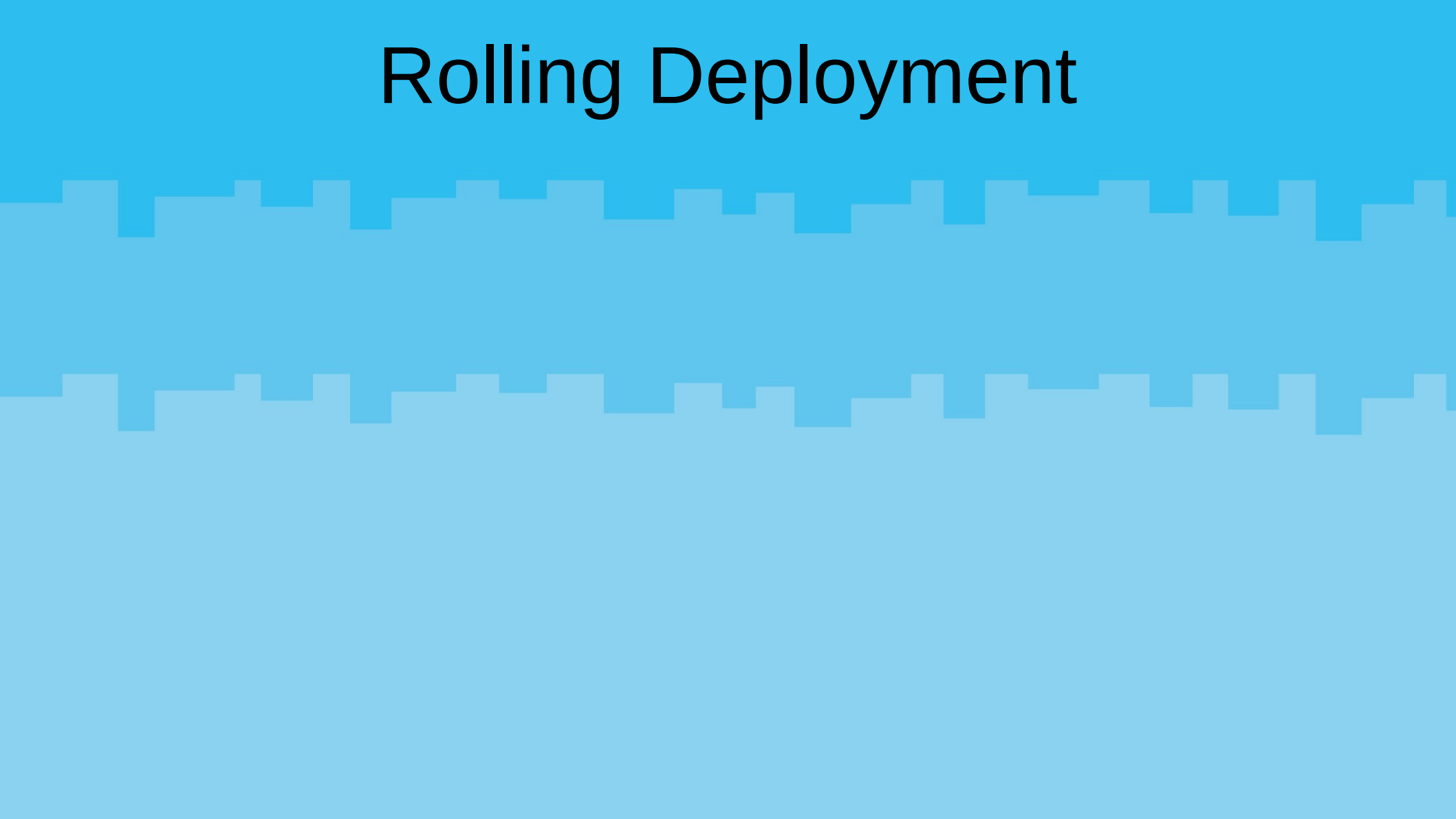


Resilience: Infrastructure-First

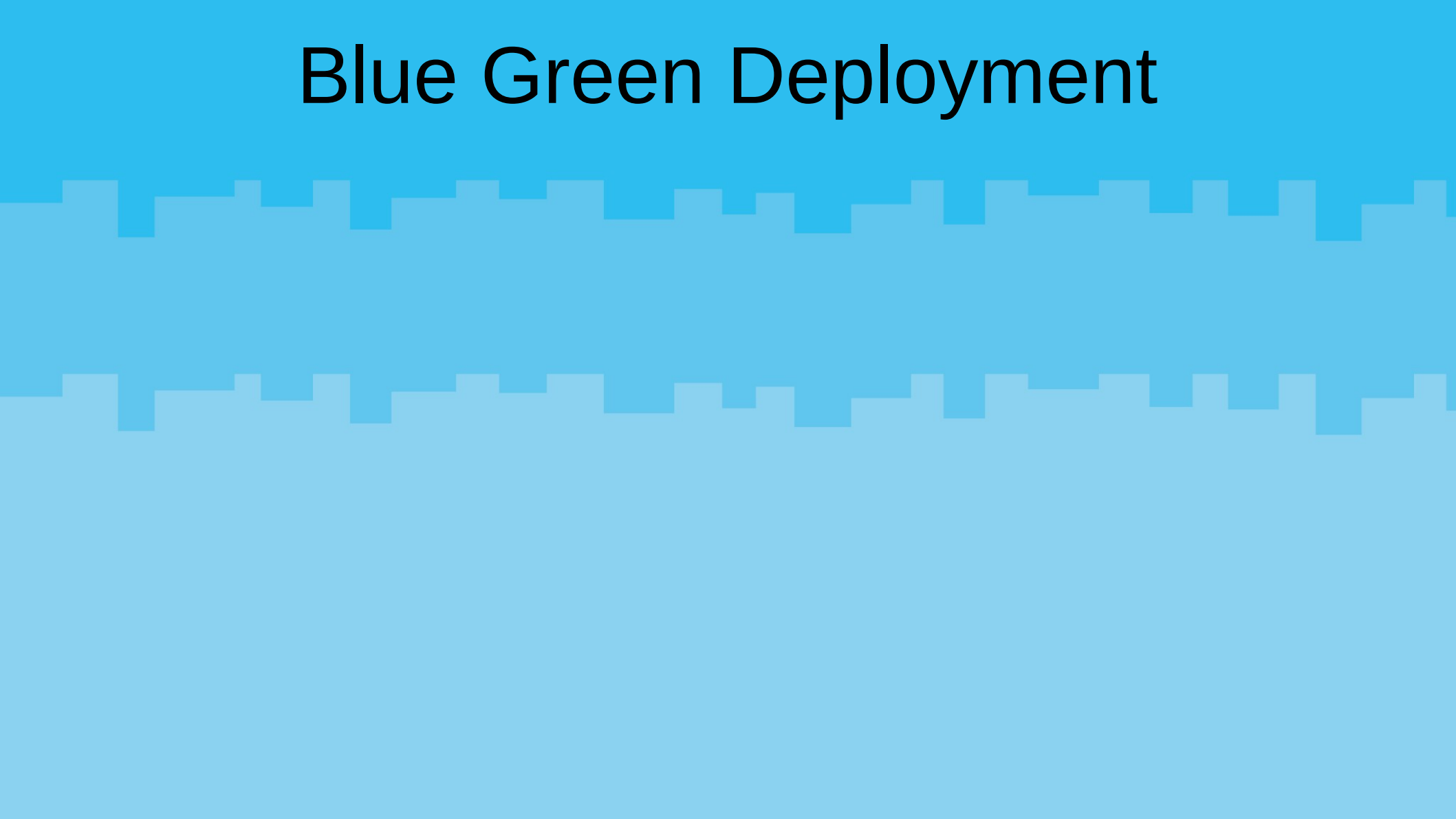
- Kubernetes in production
- The first step to testing in production is having a robust production
- The second step is automating your work
- Kubernetes leverages CNCF tools for advanced deployment features and CI/CD

Health Checks

Rolling Deployment



Blue Green Deployment

The background of the slide is a solid light blue color. Overlaid on this background is a decorative pattern of white, pixelated, rectangular shapes. These shapes are arranged in two horizontal bands, one near the top and one near the bottom, creating a digital or 'glitch' aesthetic.

Phantom Deployment

- Parallel deployment to what is served to users
- Requests are mirrored to phantom deployment
- Phantom deployment able to process real world data
- Phantom deployment monitored for issues
- Real deployment only returns traffic to users

Feature Flagging

- Env vars for features
- Allows you to test new features without changing your source code
- Toggle the flag to test a feature
- Disable it if you broke production

OpenFeature



```
OpenFeature.setProvider(new MyProvider());

const featureFlags = OpenFeature.getClient();

const withCows = await featureFlags.getBooleanValue("with-cows", false);
if (withCows) {
  res.send(cowsay.say({ text: "Hello, world!" }));
} else {
  res.send("Hello, world!");
}
```

What if you had a test environment?

- With containers, it's easy to test
- In Kubernetes, deployments are defined through yaml manifests
- Create a parallel deployment in a different namespace
- Test your deployments on the same physical servers but separated from prod